



Complete MERN Stack Interview Guide

Interview Date: Saturday, December 27, 2025

Format: MCQ + Coding Round (1 hour)



Table of Contents

Chapters

1. [JavaScript Fundamentals \(ES6+\)](#) - var/let/const, closures, promises, async/await, array methods
 2. [Node.js](#) - Event-driven architecture, core modules, NPM, streams
 3. [Express.js](#) - Routing, middleware, request/response, error handling
 4. [MongoDB & Mongoose](#) - Schemas, CRUD, operators, aggregation, population
 5. [React.js](#) - Components, hooks (useState, useEffect, useContext, etc.), forms
-



2-Day Study Plan

Day	Focus	Time	Chapters
Day 1 (Dec 25)	Backend	6-8 hrs	JavaScript, Node.js, Express.js
Day 2 (Dec 26)	Frontend & Practice	6-8 hrs	MongoDB, React.js, MCQ Practice



Key Topics for MCQs

JavaScript

- var vs let vs const (scope, hoisting)
- == vs === (type coercion)
- Closures and scope chain
- Promise states and methods
- Array methods (map, filter, reduce)
- Event loop order (sync → microtasks → macrotasks)

Node.js

- Event-driven, non-blocking I/O
- Core modules (fs, path, http)
- NPM commands and package.json

Express.js

- Middleware execution order
- HTTP methods and status codes
- Route parameters vs query strings

MongoDB

- Document vs SQL terminology
- Query operators (\$gt, \$in, \$and, etc.)
- Aggregation pipeline stages

React

- Virtual DOM and reconciliation
 - useState vs useReducer
 - useEffect dependency array
 - Controlled vs uncontrolled components
 - Keys in lists
-

Quick Reference Tables

HTTP Status Codes

Code	Meaning	When to Use
200	OK	Successful GET/PUT
201	Created	Successful POST
204	No Content	Successful DELETE
400	Bad Request	Invalid input
401	Unauthorized	No/invalid token
403	Forbidden	No permission
404	Not Found	Resource missing
500	Server Error	Unhandled error

var vs let vs const

Feature	var	let	const
Scope	Function	Block	Block
Hoisting	Yes (undefined)	TDZ	TDZ
Redeclare	✓	✗	✗
Reassign	✓	✓	✗

MongoDB vs SQL

MongoDB	SQL
Database	Database
Collection	Table
Document	Row
Field	Column
Embedded doc	Join
_id	Primary Key

React Hooks Summary

Hook	Purpose
useState	Local state
useEffect	Side effects
useContext	Global state (no prop drilling)
useRef	DOM refs, mutable values
useReducer	Complex state logic
useMemo	Memoize values
useCallback	Memoize functions

Final Checklist

JavaScript

- ☐ var/let/const differences and scoping
- ☐ Hoisting and Temporal Dead Zone
- ☐ Closures - real-world examples
- ☐ Arrow functions vs regular (this binding)
- ☐ Destructuring (array and object)
- ☐ Spread and rest operators
- ☐ Promises (then/catch, Promise.all)
- ☐ Async/await with try/catch
- ☐ Array methods (map, filter, reduce, find)
- ☐ Event loop (sync, microtask, macrotask)

Node.js

- ☐ Event-driven architecture
- ☐ Non-blocking I/O
- ☐ Core modules (fs, path, http, events)
- ☐ NPM and package.json

Express.js

- ☐ Basic server setup
- ☐ Routing with params
- ☐ Middleware (custom, error handling)
- ☐ Request and response objects
- ☐ Router for modular routes

MongoDB

- ☐ Schema definition with validators
- ☐ CRUD operations

- ☐ Query operators
- ☐ Population (refs/joins)
- ☐ Aggregation basics

React

- ☐ Components and JSX
- ☐ Props and children
- ☐ useState and state updates
- ☐ useEffect and cleanup
- ☐ useContext for global state
- ☐ useRef for DOM access
- ☐ Custom hooks
- ☐ Controlled forms
- ☐ Lists and keys
- ☐ Virtual DOM concept

Common Coding Patterns

```
// 1. Async Error Handler (Express)
const asyncHandler = fn => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

// 2. JWT Authentication
const token = jwt.sign({ id: user._id }, SECRET, { expiresIn: '7d' });
const decoded = jwt.verify(token, SECRET);

// 3. Debounce Hook (React)
function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debounced;
}

// 4. Array State Updates (React)
setItems([...items, newItem]); // Add
setItems(items.filter(i => i.id !== id)); // Remove
setItems(items.map(i => i.id === id ? {...i, done: true} : i)); // Update
```

Good luck with your interview! 🍀

Remember: Focus on understanding concepts, not just memorizing. Be ready to explain your thought process!